
MODULE *streams*

The stream layer of the version 2 *Zcash P2P* protocol (draft *ZIP*, *zcash/zips#1344*, “Stream Layer”), as realised by *QUIC*.

A connection carries many streams. Each stream is reliable and ordered on its own, but streams are mutually independent: data on one stream is never delayed by another (draft: “Transport Requirements”). The model therefore keeps one in-flight queue PER STREAM rather than one inbox per connection as the legacy model does; delivery picks any stream, which yields every cross-stream interleaving the transport allows.

Two control signals can overtake queued data in *QUIC* and are modeled as flags rather than queue elements: *RESET_STREAM* (*in_reset*, sent by the peer on its own sending direction) and *STOP_SENDING* (*stop*, the peer asking us to stop sending). *FIN* is ordered after the data it follows and is a queue sentinel.

This module defines data shapes and pure helpers only; the actions live in protocol.tla.

EXTENDS *Naturals*, *Sequences*, *FiniteSets*

CONSTANT *MaxStreams* *stream slots per opener per connection*

Stream types (draft : “Stream Types”). The type is the first byte written on every stream; the receiver learns it only by consuming that byte.

HandshakeType $\triangleq$ “0x00” *bidirectional, initiator only, exactly one*
BlockAnnType $\triangleq$ “0x10” *unidirectional, long – lived*
AnnTypes $\triangleq$ { *BlockAnnType* }
StreamTypes $\triangleq$ { *HandshakeType* } $\cup$ *AnnTypes*
BidiTypes $\triangleq$ { *HandshakeType* }

Application error codes (draft : “Application Error Codes”).

ErrorCodes $\triangleq$ { “NO_ERROR”, “PROTOCOL_ERROR”, “UNSUPPORTED_STREAM_TYPE”,
 “OBSOLETE”, “SELF_CONNECTION”, “FLOOD”, “MISBEHAVIOR”,
 “CANCELLED”, “REFUSED”, “INTERNAL_ERROR” }

NoCode $\triangleq$ “none”

Every queue element is a record with a kind field so that TLC can compare them : the type byte that opens a stream, the FIN sentinel of a finished sending direction, and the protocol records of records.tla.

TypeByte(*t*) $\triangleq$ [*kind* $\mapsto$ “type”, *t* $\mapsto$ *t*]
FIN $\triangleq$ [*kind* $\mapsto$ “fin”]

Stream identifiers for the connection between n and m : who opened it, and a slot number. Slots are reused once both peers have closed the stream.

StreamIds(*n*, *m*) $\triangleq$ { *n*, *m* } $\times$ (1 .. *MaxStreams*)

A peer s local view of one stream. Queued data flows TOWARD the owner of the record: a sender appends to the remote peer’s record, exactly as the

legacy model appends to the remote inbox.

status “none” (slot free) | “open” | “closed” (done locally, peer may not be)
 rtype stream type once known; “unknown” until the type byte is consumed
 inq data in flight toward this peer, *FIN*-terminated when finished
 in_reset error code of a *RESET_STREAM* from the peer, observable any time
 out this peer’s own sending direction: “open” | “finished” | “reset” | “na”
 stop error code of a *STOP_SENDING* from the peer for our direction

$$\text{NullStream} \triangleq [\text{status} \mapsto \text{“none”}, \\ \text{rtype} \mapsto \text{“unknown”}, \\ \text{inq} \mapsto \langle \rangle, \\ \text{in_reset} \mapsto \text{NoCode}, \\ \text{out} \mapsto \text{“na”}, \\ \text{stop} \mapsto \text{NoCode}]$$

A locally closed stream collapses to one canonical record so that the state space does not retain the history of how it was closed.

$$\text{ClosedStream} \triangleq [\text{NullStream} \text{ EXCEPT } \text{!.status} = \text{“closed”}]$$

The opener’s record of a stream it just opened with type t .

$$\text{OpenerStream}(t) \triangleq [\text{status} \mapsto \text{“open”}, \\ \text{rtype} \mapsto t, \\ \text{inq} \mapsto \langle \rangle, \\ \text{in_reset} \mapsto \text{NoCode}, \\ \text{out} \mapsto \text{“open”}, \\ \text{stop} \mapsto \text{NoCode}]$$

The remote peer’s record of the same stream: the type byte is the first queued element, followed by any records written at open time.

$$\text{PeerStream}(t, \text{payload}) \triangleq [\text{status} \mapsto \text{“open”}, \\ \text{rtype} \mapsto \text{“unknown”}, \\ \text{inq} \mapsto \langle \text{TypeByte}(t) \rangle \circ \text{payload}, \\ \text{in_reset} \mapsto \text{NoCode}, \\ \text{out} \mapsto \text{IF } t \in \text{BidiTypes} \text{ THEN “open” ELSE “na”}, \\ \text{stop} \mapsto \text{NoCode}]$$

Transport-side delivery helpers. Writes to a stream the receiver has already closed are dropped, as *QUIC* discards data after *STOP_SENDING*.

$$\begin{aligned} \text{PutData}(s, x) &\triangleq \text{IF } s.\text{status} = \text{“open”} \text{ THEN } [s \text{ EXCEPT } \text{!.inq} = \text{Append}(\text{@}, x)] \text{ ELSE } s \\ \text{PutReset}(s, \text{code}) &\triangleq \text{IF } s.\text{status} = \text{“open”} \text{ THEN } [s \text{ EXCEPT } \text{!.in_reset} = \text{code}] \text{ ELSE } s \\ \text{PutStop}(s, \text{code}) &\triangleq \text{IF } s.\text{status} = \text{“open”} \text{ THEN } [s \text{ EXCEPT } \text{!.stop} = \text{code}] \text{ ELSE } s \end{aligned}$$

Head-of-queue classification.

$$\begin{aligned} \text{IsTypeByte}(x) &\triangleq x.\text{kind} = \text{“type”} \\ \text{IsFin}(x) &\triangleq x.\text{kind} = \text{“fin”} \end{aligned}$$